Module-2

Structured Query Language

- Structured Query Language is a standard Database language that is used to create, maintain, and retrieve the relational database.
- SQL is case insensitive.
- But it is a recommended practice to use keywords (like SELECT, UPDATE, CREATE, etc.) in capital letters and use user-defined things (like table name, column name, etc.) in small letters.

What is Relational Database?

- A relational database means the data is stored as well as retrieved in the form of relations (tables).
- Table 1 shows the relational database with only one relation called **STUDENT** which stores **ROLL_NO**, **NAME**, **ADDRESS**, **PHONE,** and **AGE** of students.

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20
4	SURESH	DELHI	9156768971	18

Important Terminologies

- These are some important terminologies that are used in terms of relation.
  - **Attribute :**Attributes are the properties that define a relation.
    e.g.; **ROLL_NO**, **NAME** etc.
  - **Tuple :**Each row in the relation is known as tuple.
  - **Degree :**The number of attributes in the relation is known as degree of the relation. The **STUDENT** relation defined above has degree 5.
  - **Cardinality :**The number of tuples in a relation is known as cardinality.
    The **STUDENT** relation defined above has cardinality 4.
  - **Column :**Column represents the set of values for a particular attribute. The column **ROLL_NO** is extracted from relation STUDENT.

How Queries can be Categorized in Relational Database?

- The queries to deal with relational database can be categorized as:
  - **Data Definition Language :**It is used to define the structure of the database. e.g; CREATE TABLE, ADD COLUMN, DROP COLUMN and so on.
  - **Data Manipulation Language :**It is used to manipulate data in the relations. e.g.; INSERT, DELETE, UPDATE and so on.
  - **Data Query Language :**It is used to extract the data from the relations. e.g.; SELECT So first we will consider the Data Query Language. A generic query to retrieve data from a relational database is:
    - **SELECT** [**DISTINCT**] Attribute_List
    - **FROM** R1,R2….RM
    - [**WHERE** condition]
    - [**GROUP BY** (Attributes)[**HAVING** condition]]
    - [**ORDER BY**(Attributes)[**DESC**]];

Different Query Combinations

- **SELECT** ROLL_NO, NAME **FROM** STUDENT;
- **SELECT** ROLL_NO, NAME **FROM** STUDENT **WHERE ROLL_NO**>2;
- **SELECT * FROM** STUDENT **WHERE** ROLL_NO>2;
- **SELECT * FROM** STUDENT **ORDER BY** AGE;
- **SELECT DISTINCT** ADDRESS **FROM** STUDENT;

Aggregation Functions

- **COUNT:** Count function is used to count the number of rows in a relation. e.g;
  - **SELECT COUNT** (PHONE) **FROM** STUDENT;
- **SUM:** SUM function is used to add the values of an attribute in a relation.
  - SELECT SUM(AGE) FROM STUDENT;
- **AVERAGE:** It gives the average values of the tupples. It is also defined as sum divided by count values.
  - **Syntax:**
  - *AVG(attributename)*
  - *OR*
  - *SUM(attributename)/COUNT(attributename)*

- **MAXIMUM:** It extracts the maximum value among the set of tupples.
  - **Syntax:**
  - *MAX(attributename)*
- **MINIMUM:**It extracts the minimum value amongst the set of all the tupples.
  - **Syntax:**
  - *MIN(attributename)*
- **GROUP BY :** Group by is used to group the tuples of a relation based on an attribute or group of attribute. It is always combined with aggregation function which is computed on group.
  - **ex: SELECT** ADDRESS, **SUM**(AGE) **FROM** STUDENT
    GROUP BY (ADDRESS);

Basic Statistics with SQL

- Mean Value
  - The average of the dataset is calculated by dividing the total sum by the number of values (count) in the dataset. This operation can be performed in SQL by using built-in operation Avg.
    - SELECT Avg(ColumnName) as MEAN
      FROM TableName
- Mode Value
  - The mode is the value that appears most frequently in a series of the given data. SQL does not provide any built-in function for this, so we need to write the following queries to calculate it.
    - SELECT TOP 1 ColumnName
      FROM TableName
      GROUP BY ColumnName
      ORDER BY COUNT(*) DESC

- Median value
  - In a sorted list (ascending or descending) of numbers, the median value is the middle number and can be more descriptive of that dataset than the average.
  - For an odd amount of numeric dataset, the median value is the number that is in the middle, with the same amount of numbers below and above [2].
  - For an even amount of numbers in the list, the middle two numbers are added together, and this sum is divided by two to calculate the median value. SQL does not have a built-in function to calculate the median value of a column. The following SQL code can be used to calculate the median value [3].
    - Code to calculate the median of salary column

```
SET @rindex := -1;
SELECT
AVG(m.sal)
FROM
(SELECT @rindex:=@rindex + 1 AS rowindex,
Emp.salary AS sal
FROM Emp
ORDER BY Emp.salary) AS m
WHERE
m.rowindex IN (FLOOR(@rindex / 2) , CEIL(@rindex / 2));
```

Emp Table with Ten Employees Salary Details

salary	name
1,000	Amit
2,000	Nisha
3,000	Yogesh
4,000	Puja
9,000	Ram
7,000	Husain
8,000	Risha
5,000	Anil
10,000	Kumar
6,000	Shiv

Data Munging with SQL

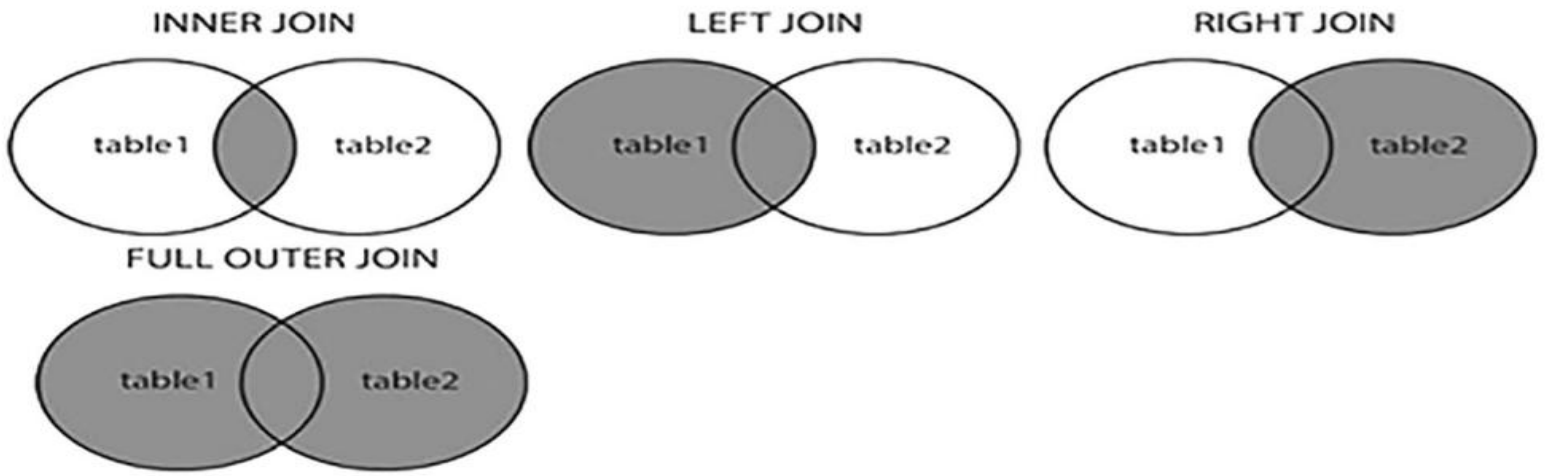
- Data munging (or wrangling) is the phase of data transformation. It transforms data into various states so that it is simpler to work and understand the data.
- The transformation may lead to manually convert or merge or update the data manually in a certain format to generate data, which is ready for processing by the data analysis tools.
- In this process, we actually transform and map data from one format to another format to make it more valuable for a variety of analytics tools.

- UPPER()
  - The UPPER() string function is used to convert the lower case to the upper case of the input text data.
    - Input: SELECT UPPER('welcome to Data Science')
    - Output: WELCOME TO DATA SCIENCE
- LOWER()
  - The LOWER() string function is used to convert the upper case to the lower case of the input text data.
    - Input: SELECT LOWER('WELCOME TO DATA SCIENCE')
    - Output: welcome to data science
- TRIM()
  - The TRIM() string function removes blanks from the leading and trailing position of the given input data.
    - Input: SELECT TRIM(' WELCOME TO DATA SCIENCE ')
    - Output: ‘WELCOME TO DATA SCIENCE’
- LTRIM()
  - The LTRIM() string function is used to remove the leading blank spaces from a given input string.
    - Input: SELECT LTRIM(' WELCOME TO DATA SCIENCE')
    - Output: ‘WELCOME TO DATA SCIENCE’

- RTRIM ()
  - The RTRIM() string function is used to remove the trailing blank spaces from a given input string.
    - Input: SELECT RTRIM('WELCOME TO DATA SCIENCE ')
    - Output: ‘WELCOME TO DATA SCIENCE’
- RIGHT()
  - The RIGHT() string function is used to return a given number of characters from the right side of the given input string.
    - Input: SELECT RIGHT('WELCOME TO DATA SCIENCE', 7)
    - Output: SCIENCE
- LEFT()
  - The LEFT() string function is used to return a given number of characters from the left side of the given input string [4].
    - Input: SELECT LEFT('WELCOME TO DATA SCIENCE', 7)
    - Output: WELCOME
- REPLACE()
  - The REPLACE() string function replaces all the occurrences of a source substring with a target substring of a given string.
    - Input: SELECT REPLACE('WELCOME TO DATA SCIENCE',
    - 'DATA', 'INFORMATION')
    - Output: WELCOME TO INFORMATION SCIENCE

Joins

- In SQL, joins are commands that are used to combine rows from two or more tables.
- These tables are combined based on a related column between those tables.
- Inner, left, right, and full are four basic types of SQL joins.
- Venn diagram is the easiest way to explain the difference between these four types.



Inner Join

- The inner join selects all rows from both tables as long as there is a match between the columns. This join returns those records that have matching values in both tables.
- So, if you perform an inner join operation between the Emp table and the Dept table, all the records that have matching values in both the tables will be given as output [5].
  - SELECT *
  - FROM Emp inner join Dept
  - on Emp.ID = Dept.ID;

TABLE 3.2

Emp Table

ID	STATE
10	AB
11	AC
12	AD

TABLE 3.3

Dept Table

ID	BRANCH
11	Computer
12	Civil
13	Mech

Result after Inner Join

ID	STATE	ID	BRANCH
11	AC	11	Computer
12	AD	12	Civil

Left Outer Join

- The left outer join (or left join) returns all the rows of the left side table and matching rows in the right-side table of the join.
- The rows for which there is no matching row on the right side, the result will contain NULL.
- From the left table (Emp), the left join keyword returns all records, even if there are no matches in the right tables (Dept).
  - SELECT *
  - FROM Emp left outer join Dept
  - on Emp.ID = Dept.ID;

Result Table after Left Join

ID	STATE	ID	BRANCH
10	AB	NULL	NULL
11	AC	11	Computer
12	AD	12	Civil

Right Outer Join

- The right outer join (or right join) returns all the rows of the right side table and matching rows for the left side table of the join.
- The rows for which there is no matching row on the left side, the result will contain NULL.
  - SELECT *
  - FROM Emp right outer join Dept
  - on Emp.ID = Dept.ID;

Result Table after Right Outer Join

ID	STATE	ID	BRANCH
11	AC	11	Computer
12	AD	12	Civil
NULL	NULL	13	Mech

Full Outer Join

- The full outer join (or full join) returns all those records that have a match in either the left table (Emp) or the right table (Dept) table.
- The joined table will contain all records from both the tables, and if there is no matching field on either side, then fill in NULLs.
  - SELECT *
  - FROM Emp full outer join Dept
  - on Emp.ID = Dept.ID;

Result Table after Full Outer Join

ID	STATE	ID	BRANCH
10	AB	NULL	NULL
11	AC	11	Computer
12	AD	12	Civil
NULL	NULL	13	Mech

An organization maintains two database tables to manage **employee master data** and **project assignment details**.
However, not all employees are assigned to projects, and some project records may refer to employees who are no longer active.

Table 1: EMPLOYEE

Emp_ID	Emp_Name	Department	Designation	Salary	Location
E101	Arun	CSE	Analyst	55000	Chennai
E102	Divya	ECE	Engineer	60000	Bangalore
E103	Karthik	IT	Developer	65000	Hyderabad
E104	Meena	CSE	Manager	80000	Chennai
E105	Rahul	EEE	Engineer	58000	Pune
E106	Sneha	IT	Analyst	54000	Kochi
E107	Ajay	Civil	Supervisor	50000	Coimbatore

Table 2: PROJECT_ASSIGNMENT

Project_ID	Emp_ID	Project_Name	Start_Date	End_Date	Project_Status
P301	E101	Smart Traffic	01-01-2024	30-06-2024	Completed
P302	E103	AI Chatbot	15-02-2024	15-08-2024	Ongoing
P303	E104	ERP System	01-03-2024	30-09-2024	Ongoing
P304	E108	Solar Grid	10-01-2024	10-07-2024	Completed
P305	E102	5G Network	05-02-2024	05-10-2024	Ongoing
P306	E109	Smart Parking	01-04-2024	31-12-2024	Planned
P307	E105	Power Automation	20-01-2024	20-06-2024	Completed

- Display the details of all employees working in the CSE or IT departments whose salary is greater than ₹55,000, along with their project details only if:
  - the project status is Ongoing
  - and the project started after 01-02-2024
  - Employees who satisfy the department and salary conditions but do not have any matching project should still appear in the result, with NULL values in project-related columns.

Answer

```
SELECT
    E.Emp_ID,
    E.Emp_Name,
    E.Department,
    E.Designation,
    E.Salary,
    E.Location,
    P.Project_ID,
    P.Project_Name,
    P.Start_Date,
    P.End_Date,
    P.Project_Status
FROM EMPLOYEE E
LEFT OUTER JOIN PROJECT_ASSIGNMENT P
    ON E.Emp_ID = P.Emp_ID
    AND P.Project_Status = 'Ongoing'
    AND P.Start_Date > '2024-02-01'
WHERE
    E.Department IN ('CSE', 'IT')
    AND E.Salary > 55000;
```

Aggregation

SQL provides the following built-in functions for aggregating data.

- The COUNT() function returns the number of values in the dataset (“salary” is column in table “Emp”).
  - SELECT COUNT(salary)
  - FROM Emp

EmpID	EmpName	Department	JobTitle	Salary	Bonus	Experience	City
101	Arun	IT	Developer	60000	5000	3	Chennai
102	Priya	HR	Manager	75000	8000	7	Bangalore
103	Karthik	IT	Developer	65000	6000	4	Chennai
104	Meena	Finance	Analyst	55000	4000	2	Mumbai
105	Ravi	IT	Team Lead	90000	10000	10	Hyderabad
106	Sneha	HR	Executive	45000	3000	2	Chennai
107	Vijay	Finance	Manager	80000	9000	8	Delhi

- The AVG() function returns the average of a group of selected numeric column values
  - SELECT AVG(salary)
  - FROM Emp
- The SUM() function returns the total sum of a numeric column.
  - SELECT SUM(salary)
  - FROM Emp
- The MIN() function returns the minimum value in the selected column.
  - SELECT MIN(salary)
  - FROM Emp
- The MAX() function returns the maximum value in the selected column.
  - SELECT MAX(salary)
  - FROM Emp

Filtering

- The data generated from the reports of various application software often results in complex and large datasets.
- This dataset may consist of redundant records or impartial records. This useless data may confuse the user.
- Filtering this redundant and useless data can also make the dataset more efficient and useful. Data filtering is one of the major steps involved in data science due to various reasons, and some are listed below:
  - During certain situations, we may require a specific part of the actual data for analysis.
  - Sometimes, we may require reducing the actual retrieved data by removing redundant records as that may result in wrong analysis.
  - Query performance can be greatly enhanced by applying it to refined data. Also, it can reduce strain on application.
  - Data filtering process consists of different strategies for refining and reducing datasets.

- Syntax to filter data using WHERE
  - SELECT *
  - FROM tablename;
  - Example:
    - SELECT *
    - FROM tablename
    - WHERE columnname = expected_value;
  - we can use the following operators too:
    - > (greater than),
    - < (less than),
    - >= (greater than or equal to),
    - (less than or equal to), and <=
    - != (not equal to).

- The LIKE clause can be used to specify a pattern matching condition.
- Two wildcards, percent sign “%” and underscore “_”, are used to specify conditions.
- The percent sign is used to represent any string of zero or more characters, and underscore represents a single number or character.
- For example, to retrieve ENAME that ends with character “y” of workers table, we can write the query as follows:
    - select ENAME
    - from workers
    - where ENAME like ‘%a';

- select SALARY
- from workers
- where SALARY like '_5%’;

- The SQL IN operator allows you to test if the given expression matches any value in the list of values. If the records matched with any one of the values in the list, then it is returned as result. For example,
  - select *
  - from workers
  - where DEPTNAME IN(‘HR', ‘IT’);

  - select *
  - from workers
  - where DEPTNAME NOT IN('Testing', 'Workshop');

EmpID	EmpName	Department	JobTitle	Salary	Bonus	Experience	City
101	Arun	IT	Developer	60000	5000	3	Chennai
102	Priya	HR	Manager	75000	8000	7	Bangalore
103	Karthik	IT	Developer	65000	NULL	4	Chennai
104	Meena	Finance	Analyst	55000	4000	2	Mumbai
105	Ravi	IT	Team Lead	90000	10000	10	Hyderabad
106	Sneha	HR	Executive	45000	3000	2	Chennai
107	Vijay	Finance	Manager	80000	9000	8	Delhi
108	Divya	IT	Tester	50000	3500	3	Bangalore
109	Manoj	Marketing	Executive	48000	NULL	2	Mumbai
110	Anjali	Marketing	Manager	70000	7000	6	Hyderabad
111	Rahul	IT	Developer	62000	5500	4	Delhi

- Write a SQL query to:
  - Display Department,
  - Total number of employees in each department,
  - Average salary,
  - Total compensation (Salary + Bonus, treating NULL bonus as 0),
- Only for departments that:
  - Have at least 2 employees,
  - Have average salary greater than the overall company average salary,
  - And whose minimum salary is greater than 50000.
  - Sort the result in descending order of total compensation.

```
SELECT
    Department,
    COUNT(*) AS TotalEmployees,
    AVG(Salary) AS AvgSalary,
    SUM(Salary + COALESCE(Bonus, 0)) AS TotalCompensation
FROM Emp
GROUP BY Department
HAVING COUNT(*) >= 2
   AND AVG(Salary) > (SELECT AVG(Salary) FROM Emp)
   AND MIN(Salary) > 50000
ORDER BY TotalCompensation DESC;
```

- Using the Emp table given previously, write a SQL query to:
  - Display EmpID, EmpName, Department, Salary, City
  - Include only employees:
    - Whose name starts with 'A' or ends with 'a' (LIKE)
    - AND who belong to departments IT, HR, or Finance (IN)
    - AND who are not working in cities Mumbai or Delhi (NOT IN)
  - Additionally, include only employees whose salary is greater than 55000
  - Sort the output by Salary in descending order

Window Functions and Ordered Data

- In SQL window functions, the word “window” refers to a set of rows. It is similar to aggregate functions, but when we use the GROUP BY clause with aggregate functions, it groups the result set based on one or more columns.
- The aggregate function is performed on the rows as an entire group.
- SQL window functions generate a result with some attributes of an individual row together with the results of the window function.
- Window functions can be called with the SELECT statement or the ORDER BY clause, but cannot be called in the WHERE clause.
- The window function calculates on a set of rows and returns a value for each row from the given query. In the window function, the term window represents the set of rows on which the function operates.
- It calculates the returned values based on the values of the rows in a window

- The OVER() clause has the following subclauses:
  - PARTITION BY clause to define window partitions to form groups of rows on which window function will be applied.
  - ORDER BY clause for logical sorting of rows within a partition.

RANK () Window Function

- The RANK() function returns the position of any row in the specified partition.
- The OVER and PARTITION BY functions are used to divide the result set into partitions according to specified criteria. Further, ORDER BY clause can be used to sort data in ascending or descending order based on some attribute.
- To rank salaries within departments, we can write the query as follows:

```
SELECT
                RANK() OVER (PARTITION BY DEPTNAME ORDER BY
SALARY DESC)
                        AS DEPT_RANK,
                DEPTNAME,
                DEPTID,
                SALARY,
                ENAME,
                EID
FROM workers;
```

"workers" Table

ENAME	EID	SALARY	DEPTID	DEPTNAME
John	11	30,000	301	Workshop
Jerry	15	35,000	305	Testing
Niya	38	45,000	308	HR
Alice	18	45,000	305	Testing
Tom	24	50,000	301	Workshop
Bobby	17	58,000	308	HR
Reyon	16	30,000	305	Testing
Bob	22	51,000	301	Workshop

Result Table

DEPT_RANK	DEPTNAME	DEPTID	SALARY	ENAME
1	HR	308	58,000	Bobby
2	HR	308	45,000	Niya
1	Testing	305	45,000	Alice
2	Testing	305	35,000	Jerry
3	Testing	305	30,000	Reyon
1	Workshop	301	51,000	Bob
2	Workshop	301	50,000	Tom
3	Workshop	301	30,000	John

PERCENT_RANK()

- In SQL Server, the PERCENT_RANK() function calculates the SQL percentile rank of each row.
- This percentile ranking number may range from zero to one.
- For each row, PERCENT_RANK() calculates the percentile rank using the following formula:

$$(\text{Rank} - 1)/(\text{total\_number\_rows} - 1)$$

- In this formula, rank represents the rank of the row. total_number_rows is the number of rows that are being evaluated.
- It always returns the rank of the first row as 0.

```
SELECT
                PERCENT_RANK() OVER (PARTITION BY DEPTNAME ORDER
BY SALARY DESC)
```

Result Table

DEPT_RANK	DEPTNAME	DEPTID	SALARY	ENAME	EID
0	HR	308	58,000	Bobby	17
1	HR	308	45,000	Niya	38
0	Testing	305	45,000	Alice	18
0.5	Testing	305	35,000	Jerry	15
1	Testing	305	30,000	Reyon	16
0	Workshop	301	51,000	Bob	22
0.5	Workshop	301	50,000	Tom	24
1	Workshop	301	30,000	John	11

```
SELECT
                PERCENT_RANK() OVER (PARTITION BY DEPTNAME ORDER
BY SALARY DESC)
                                AS DEPT_RANK,
                    DEPTNAME,
                    DEPTID,
                    SALARY,
                    ENAME,
                    EID
 FROM workers;
 SELECT
                    PERCENT_RANK() OVER (ORDER BY SALARY DESC)
                                AS DEPT_RANK,
                    DEPTNAME,
                    DEPTID,
                    SALARY,
                    ENAME,
                    EID
 FROM workers;
```

Result Table

DEPT__PERCENT_RANK	DEPTNAME	DEPTID	SALARY	ENAME	EID
0	HR	308	58,000	Bobby	17
0.14285714285714285	Workshop	301	51,000	Bob	22
0.2857142857142857	Workshop	301	50,000	Tom	24
0.42857142857142855	HR	308	45,000	Niya	38
0.42857142857142855	Testing	305	45,000	Alice	18
0.7142857142857143	Testing	305	35,000	Jerry	15
0.8571428571428571	Workshop	301	30,000	John	11
0.8571428571428571	Testing	305	30,000	Reyon	16

DENSE_RANK ()

- The DENSE_RANK () window function calculates the rank of value in a group of rows based on the ORDER BY expression specified in the OVER clause.
- For each partition, rank starts from 1. Rows with the same values receive the same rank.
- DENSE_RANK function does not keep gaps in ranks if there is a similarity between previous one or more rows ranks. This feature makes it different from RANK() function.

```
SELECT
                DENSE_RANK() OVER (ORDER BY SALARY DESC)
                        AS DEPT__DENSE_RANK,
                DEPTNAME,
                DEPTID,
                SALARY,
                ENAME,
                EID
FROM   workers;
```

Result Table

DEPT__DENSE_RANK	DEPTNAME	DEPTID	SALARY	ENAME	EID
1	HR	308	58,000	Bobby	17
2	Workshop	301	51,000	Bob	22
3	Workshop	301	50,000	Tom	24
4	HR	308	45,000	Niya	38
4	Testing	305	45,000	Alice	18
5	Testing	305	35,000	Jerry	15
6	Workshop	301	30,000	John	11
6	Testing	305	30,000	Reyon	16

NTILE()

- The SQL NTILE() function partitions a logically ordered dataset into a number of buckets demonstrated by the expression and allocates the bucket number to each row.
- The buckets are numbered from 1 through expression where the expression value must result in a positive integer value for each partition.
- Divide rows by bucket to perform group split-up.
- For example, the following query will allocate rows to three buckets
  - Total rows=8
  - Bucket=3
  - Hence 8/3= 2 remainder 2
- This means:
- Each bucket gets at least 2 rows
- 2 extra row needs to be distributed

```
SELECT
                ENAME,
                EID,
                DEPTID,
                DEPTNAME,
                SALARY,
                NTILE(3) OVER (PARTITION BY DEPTNAME ORDER BY
SALARY)
                        AS BUCKETS
FROM   workers;
```

Result Table

ENAME	EID	DEPTID	DEPTNAME	SALARY	PREVIOUS_PERSON_SALARY
Niya	38	308	HR	45,000	1
Bobby	17	308	HR	58,000	2
Reyon	16	305	Testing	30,000	1
Jerry	15	305	Testing	35,000	2
Alice	18	305	Testing	45,000	3
John	11	301	Workshop	30,000	1
Tom	24	301	Workshop	50,000	2
Bob	22	301	Workshop	51,000	3

CUME_DIST()

- The SQL window function CUME_DIST() returns the cumulative distribution of a value within a partition of values. And there is no ORDER BY in the outer query.
- So SQL is free to return partitions in alphabetical order of DEPTNAME (or whatever internal order the engine chooses).
- The cumulative distribution of a value calculated by the number of rows with values less than or equal to (<=) the current row's value is divided by the total number of rows.

$$CUME\_DIST = \frac{\text{Number of rows with value} \leq \text{current row}}{\text{Total rows in partition}}$$

For Niya (45000)

Rows ≤ 45000 = 1

Total rows = 2

$$1/2 = 0.5$$

ENAME	EID	DEPTID	DEPTNAME	SALARY	CUME_DIST_VALUE
Niya	38	308	HR	45,000	0.5
Bobby	17	308	HR	58,000	1
Reyon	16	305	Testing	30,000	0.3333333333333333
Jerry	15	305	Testing	35,000	0.6666666666666666
Alice	18	305	Testing	45,000	1
John	11	301	Workshop	30,000	0.3333333333333333
Tom	24	301	Workshop	50,000	0.6666666666666666
Bob	22	301	Workshop	51,000	1

where N is the number of rows with the value less than or equal to the current row value and total rows is the number of rows in the group or result set. Function returns value having a range between 0 and 1 (Table 3.29).

```
SELECT
                ENAME,
                EID,
                DEPTID,
                DEPTNAME,
                SALARY,
                CUME_DIST() OVER (PARTITION BY DEPTNAME ORDER BY
SALARY)
                        AS CUME_DIST_VALUE
FROM   workers;
```

ROW_NUMBER()

- The SQL window function ROW_NUMBER() is used to display a row number for each row within a specified partition

```
SELECT
        ROW_NUMBER() OVER (PARTITION BY DEPTNAME ORDER BY
SALARY DESC) AS   ROW_NUM,
              DEPTNAME,
              DEPTID,
              SALARY,
              ENAME,
              EID
FROM workers;
```

Result Table

ROW_NUM	DEPTNAME	DEPTID	SALARY	ENAME	EID
1	HR	308	58,000	Bobby	17
2	HR	308	45,000	Niya	38
1	Testing	305	45,000	Alice	18
2	Testing	305	35,000	Jerry	15
3	Testing	305	30,000	Reyon	16
1	Workshop	301	51,000	Bob	22
2	Workshop	301	50,000	Tom	24
3	Workshop	301	30,000	John	11

LEAD()

- SQL LEAD() function has a capacity that gives admittance to a column at a predefined actual counterbalance which follows the current row.
- For example, by utilizing the LEAD() function, from the current line, you can get information of the following line, or the second line that follows the current line, or the third line that follows the current line, etc.

```
SELECT
                ENAME,
                EID,
                DEPTID,
                DEPTNAME,
                SALARY,
                LEAD(SALARY) OVER (PARTITION BY DEPTNAME ORDER BY
SALARY)-SALARY
                        AS SALARY_DIFFERENCE
FROM workers;
```

Result Table

ENAME	EID	DEPTID	DEPTNAME	SALARY	SALARY_DIFFERENCE
Niya	38	308	HR	45,000	13,000
Bobby	17	308	HR	58,000	NULL
Reyon	16	305	Testing	30,000	5,000
Jerry	15	305	Testing	35,000	10,000
Alice	18	305	Testing	45,000	NULL
John	11	301	Workshop	30,000	20,000
Tom	24	301	Workshop	50,000	1,000
Bob	22	301	Workshop	51,000	NULL

LAG()

- We can use a SQL window function LAG() to access previous row’s data based on defined offset value. It works similar to a LEAD() function.
- In the SQL LEAD() function, we access the values of subsequent rows, but in LAG() function, we access previous row’s data.
- It is useful to compare the current row value from the previous row value.

```
SELECT
                ENAME,
                EID,
                DEPTID,
                DEPTNAME,
                SALARY,
                LAG(SALARY) OVER (PARTITION BY DEPTNAME ORDER BY
SALARY)
                                AS PREVIOUS_PERSON_SALARY
FROM   workers;
```

Result Table

ENAME	EID	DEPTID	DEPTNAME	SALARY	PREVIOUS_PERSON_SALARY
Niya	38	308	HR	45,000	NULL
Bobby	17	308	HR	58,000	45,000
Reyon	16	305	Testing	30,000	NULL
Jerry	15	305	Testing	35,000	30,000
Alice	18	305	Testing	45,000	35,000
John	11	301	Workshop	30,000	NULL
Tom	24	301	Workshop	50,000	30,000
Bob	22	301	Workshop	51,000	50,000

```
SELECT *,
    CASE
        WHEN quantity >= 10 THEN 'More'
        WHEN quantity >= 6 THEN 'Avg'
        ELSE 'Less'
    END AS summary
```

TABLE 3.42

"sales" Table

sale_no	product_id	quantity	price	customer_name
5,001	3	4	21,000	John
5,002	11	NULL	17,000	Anna
5,003	94	10	105,000	Tom
5,004	86	8	27,000	Nora
5,005	88	18	8,000	Tom

```
FROM
    sales;
```

Result Table

sale_no	product id	quantity	price	customer_name	summary
5,001	3	4	21,000	John	Less
5,002	11	NULL	17,000	Anna	Less
5,003	94	10	105,000	Tom	More
5,004	86	8	27,000	Nora	Avg
5,005	88	10	8,000	Tom	More

COALESCE

- Some records of database may consist of NULL values, but while applying statistics to these datasets, you may need to replace these NULL values with some other data.
- This can be done effectively by the COALESCE function.
- The first parameter to this function is a column that may consist of NULL and the second represents value that replaces NULL.
- It replaces all NULL values specified in column by the second default value given in the function.

```
SELECT
customer_name ,product_id,
COALESCE(quantity, -1) AS quantity
FROM
    sales;
```

Result Table

customer_name	product_id	quantity
John	3	4
Anna	11	–1
Tom	94	10
Nora	86	8
Tom	88	10

NULLIF

- NULLIF function takes two parameters and will return NULL if the first parameter value equals the second value else returns the first parameter.

LEAST/GREATEST

- The LEAST and GREATEST are frequently used functions for data cleaning.
- These functions return the least and greatest values from the given set of elements, respectively.
- These functions are useful to replace value in list, especially if it is too high or low.

DISTINCT

- The DISTINCT keyword returns only distinct values in the specified column value sets.

Advanced NoSQL for Data Science

- NoSQL is a database design that can accommodate various data models, including key-value, document, columnar, and graph formats.
- NoSQL, which means “not only SQL”, is an alternative to relational databases in which data is stored in tables and has a fixed data schema.
- NoSQL databases are found to be very useful for handling really big data tasks because it follows the Basically Available, Soft State, Eventual Consistency (BASE) approach instead of Atomicity, Consistency, Isolation, and Durability – commonly known as ACID properties

Features:

- It is not using the relational model to store data.
- NoSQL running well on clusters.
- It is mostly open-source.
- NoSQL is capable to handle a large amount of social media data.
- NoSQL is schema-less.

Document Databases for Data Science

- Document-based NoSQL databases store the data in the JSON object format.
- Each document has key-value pairs like structures. The document-based NoSQL databases are simple for engineers as they map items as a JSON object.
- NoSQL databases are CouchDB, MongoDB, OrientDB, and BaseX.

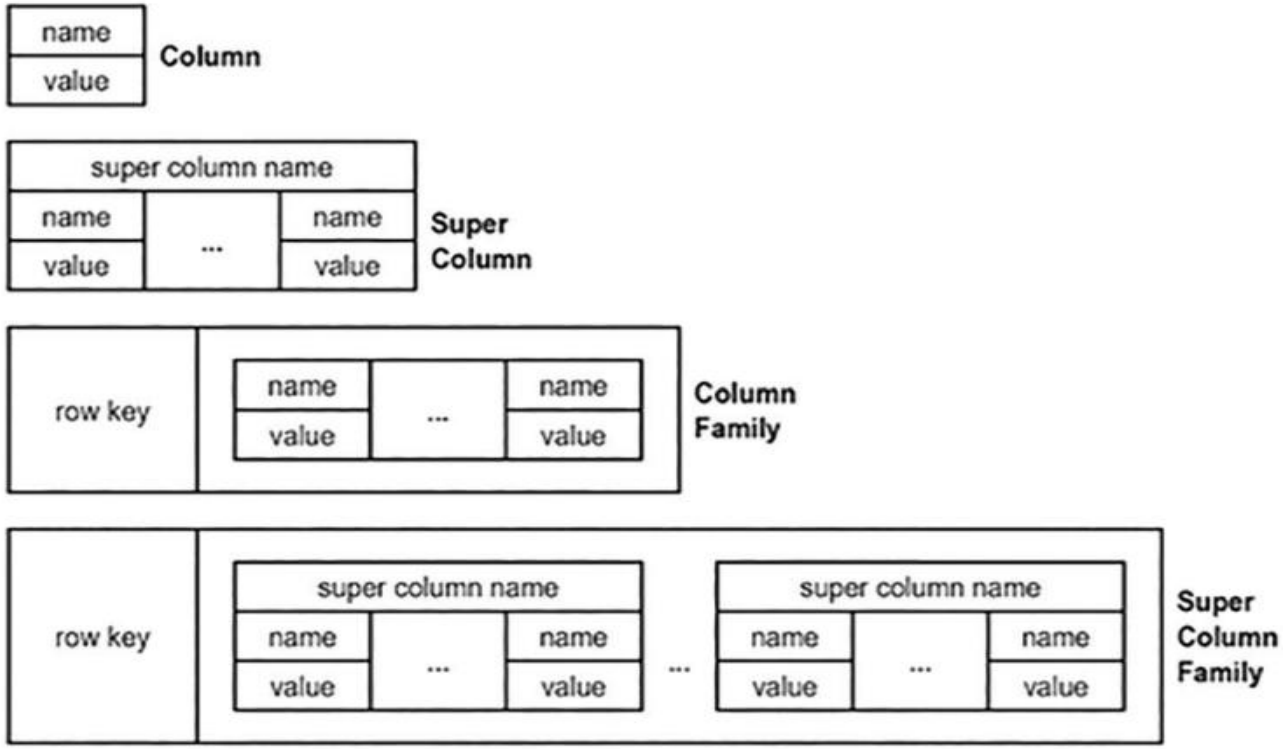
JSON document format:

```
{
   "_id": 1,
   "name" : { "first" : "John", "last" : "Backus" },
   "contribs" : [ "Fortran", "ALGOL", "Form", "FP" ],
   "awards" : [
      {
         "award" :"Dowell Award",
         "year" : 1988,
         "by" :"Computer Society"
      },
{
         "award" :"First Prize",
         "year" : 1993,
         "by" : "National Academy of Engineering"
      }
   ]
}
```

Wide-Column Databases for Data Science

- Similar to any relational database, this wide-column database stores the data in records, but it can also store very large numbers of dynamic columns.
- It groups the dynamically added columns into column families. Instead of having multiple tables like relational databases, we have multiple column families in wide-column databases.
- Examples of wide-column types of

databases are Cassandra and HBase



Graph Databases for Data Science

- Graph database stores the data in the form of nodes and edges. The node stores information about the main entities like people, places, and products, and the edge stores the relationships between them.
- Graph database is very useful to find out the pattern or relationship among data like a social network and recommendation engines. Examples of graph databases are Neo4j and Amazon Neptune